

Code of the Day

- What does the value of “f” tell?

```
int v;  
bool f = ((v & (v-1)) == 0);
```

128	1	0	0	0	0	0	0	0
127	0	1	1	1	1	1	1	1

ECE 759

High Performance Computing for Applications in Engineering

[Spring 2025]

Time and Memory Hierarchy

02/03/2025

Before we get started...

- Quick overview, things discussed last time
 - Cover registers. Store information & there's no faster way to access info than from registers
 - Instruction-level parallelism (ILP)
 - Thread-level parallelism (TLP)
- Purpose of today's lecture:
 - Timing the execution of a program
 - Memory aspects: memory hierarchy, caches (what they are, why do we need them)
- Miscellaneous
 - PA #1 is due 2/3 (today) at 23:59 PM via Canvas
 - PA #2 is due 2/17 23:59 PM via Canvas

Measuring Speed of Execution

Terminology (1/4)

- Wall Clock (Real) Time

- Amount of time from the beginning of a program to the end of the program (end-to-end runtime)
- Includes all the housekeeping work that CPU does while running our program
- Often the most meaningful indicator of performance since this is what you see

- CPU Execution Time

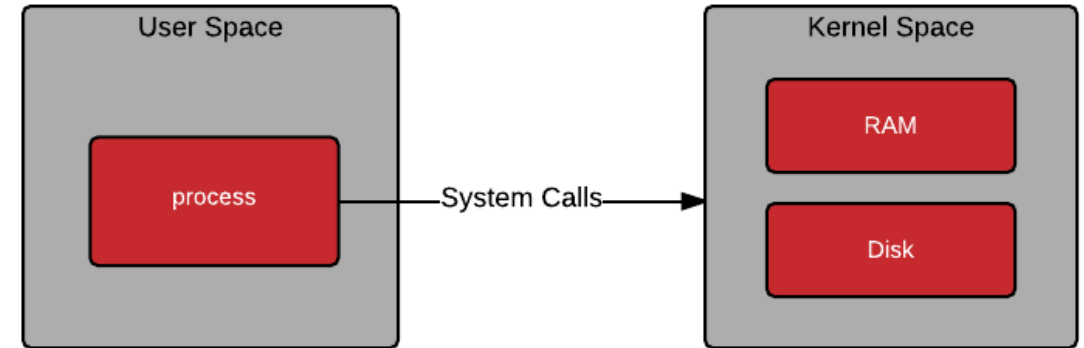
- Counts only the amount of time that is effectively dedicated to **your** program
 - Why? In reality, many other programs running on your machine at the same time (time slicing)
 - So, your operating system does a very hard and good job to multi-task multiple programs with time slicing
- Requires a profiling tool to assess (like gprof, for instance)

- On a quiet machine, Wall Clock Time and CPU Execution Time should be identical

Terminology (2/4)

- Further qualification of CPU Execution Time:

- **User time** – the amount of time spent in user-mode
- **System time** – the amount of time spent in the kernel within the process
 - Ex: open file for writing/reading, throw an exception, etc.



```
File Edit View Search Terminal Help
naman@root:~$ time -p sleep 3
real 3.00
user 0.00
sys 0.00
naman@root:~$
```

Terminology (3/4)

- Why bother to delineate “system vs. user”?
 - We can only cut into the “user time”
 - Why? Because this is the part that comes directly from your source code
 - If your code has 85% non-user time, there’s not much you can do to speed up your code
 - Why? Because you can’t really touch and rewrite those system kernel code – unless you are interested in modifying the kernel source code and recompile it for your OS
 - Ex: networking applications with many delays on waiting for internet packets to arrive – there is no way for you to rewrite the router’s code to make it run faster or so

Terminology (4/4)

- Clock cycle sets the pace at which the execution proceeds
 - A 1 GHz processor has clock cycle of 1 nanosecond
 - A 2 GHz processor has clock cycle of 0.5 nanoseconds (500 picoseconds)
- The speed of executing an instruction is decided by the clock cycle
 - Ex: an instruction takes four clock cycles on a 1 GHz processor can finish in 4 nanosecond
 - Ex: an instruction takes four clock cycles on a 2 GHz processor can finish in 2 nanosecond

The CPU Performance Equation

- Three ingredients control the amount of time for a program execution:
 - Number of instructions that your program has (Instruction Count)
 - Average number of “clock-cycles per instructions” (CPI)
 - Clock Cycle Time
- The CPU Performance Equation: $CPU\ Execution\ Time = Instruction\ Count \times CPI \times Clock\ Cycle\ Time$
- Alternatively, using the clock rate

$$CPU\ Execution\ Time = \frac{Instruction\ Count \times CPI}{Clock\ Rate}$$

CPU Performance: How can we improve it?

$$\text{CPU Execution Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

- To improve performance the product of three factors should be reduced
- For a long time, we've been on "let's increase the frequency"; i.e., reduce **Clock Cycle Time**
 - How? Just buy more expensive hardware that can power the program
 - We eventually hit a wall this way (the "Power Wall")
- As repeatedly demonstrated in practice, reducing the **Instruction Count** (IC) often leads to an increase in **CPI**. And the other way around

Side Trip: Revisiting, RISC or CISC?

- RISC: reduced instruction set computer
 - Simpler to work with but requires a large number of instructions to accomplish something in your C code
 - High IC but low CPI

$$\text{CPU Execution Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$


- CISC: complex instruction set computer
 - Complex to work with but takes only a few instructions to accomplish the same thing in your C code
 - Low IC but high CPI

$$\text{CPU Execution Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$


- Specific example: ARM vs. x86

Standard Performance Evaluation Corp (SPEC) CPU Benchmarks

- SPEC benchmarks help us understand what processor does well and what doesn't
- Idea: gather a collection of programs that use a good mix of instructions to evaluate a chip
- The “collection of programs”:
 - Should be non-trivial and widely used
 - Should be as objective as possible: not favor a chip manufacturer at the expense of another one
 - Example: a compiler source is used extensively, so it makes sense to have it included in the benchmark
- Two common categories:
 - Tied to programs dominated by floating point operations: CFP2006
 - Tied to programs dominated by integer arithmetic: CINT2006

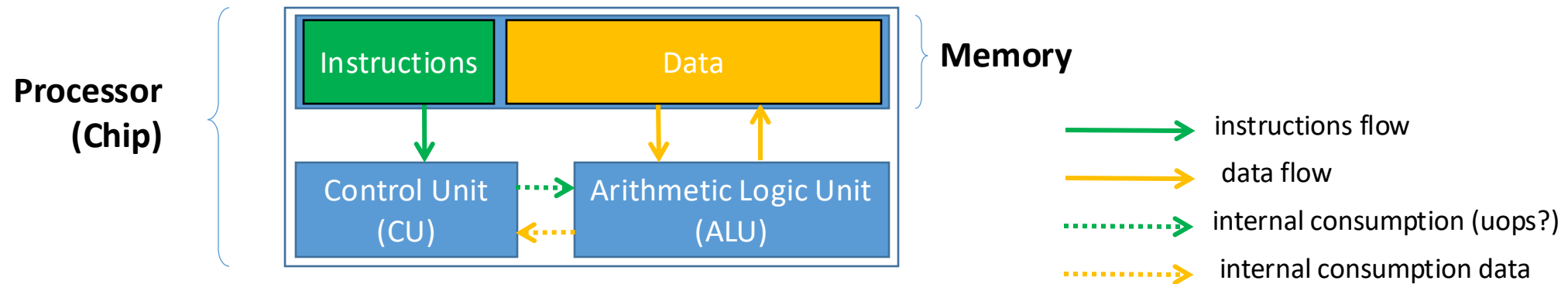
SPEC CPU Benchmark: Example, highlights AMD performance

CINT2006 Programs		AMD Opteron X4 – 2356 (Barcelona)			
Description	Name	Instruction count [$\times 10^9$]	CPI	Clock Cycle Time ns = [seconds $\times 10^{-9}$]	CPU Exec. Time [seconds]
Interpreted string processing	perl	2118	0.75	0.4	637
Block-sorting compression	bzip2	2389	0.85	0.4	817
GNU C compiler	gcc	1050	1.72	0.4	724
Combinational optimization	mcf	336	10.00	0.4	1,345
Go game (AI)	go	1658	1.09	0.4	721
Search gene sequence	hmmer	2783	0.80	0.4	890
Chess game (AI)	sjeng	2176	0.96	0.4	837
Quantum computer simulation	libquantum	1623	1.61	0.4	1,047
Video compression	h264avc	3102	0.80	0.4	993
Discrete event simulation library	omnitpp	587	2.94	0.4	690
Games/path finding	aster	1082	1.79	0.4	773
XML parsing	xatancbmk	1058	2.70	0.4	1,143

The lower the CPI, the better the performance

Big New Topic: Memory

Transistors: why we love them & want as many as possible_[old slide]

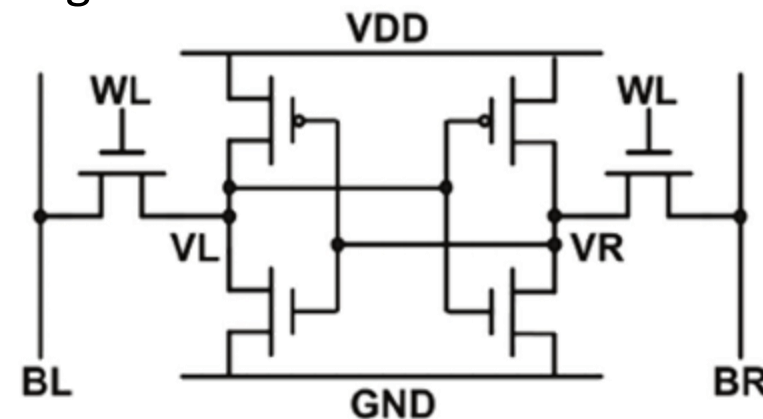


- So far, we have focused on how CU and ALU are used to **execute machine instructions**
 - CU in charge of making decision based on decoded information from an instruction
 - ALU in charge of performing the actual job based on the decision made by CU
- Next, we focus on how memory is organized to **store data and instructions**

Type Memory: SRAM

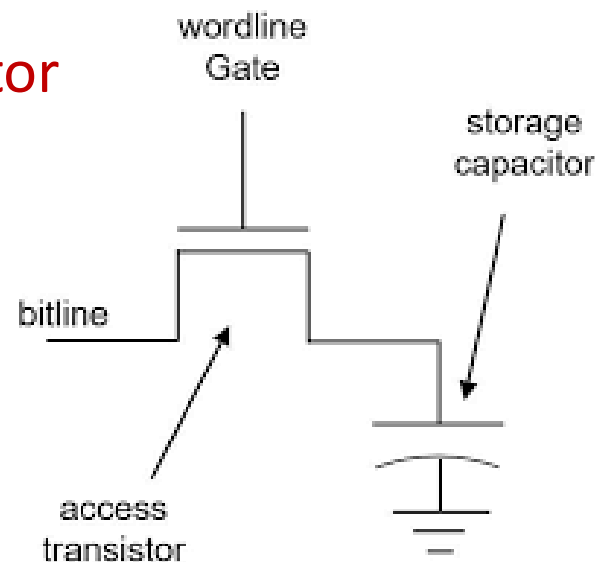
- SRAM – Static Random Access Memory

- SRAM: Integrated circuit whose elements combine to make up memory arrays
- An SRAM “element”: a special circuit, called **flip-flop**, that saves the state **one bit**
- “Static” refers to the fact that once set, the element stores the value set as long as the element is powered
- One flip-flop made of **four transistors** & requires two more access transistors: grand total – **6 transistors**
- Very **short access time**: $\approx \mathbf{O}(0)$ ns (order of magnitude(0): $10^0 = 1$)
- Uniform access time of any element in the array (yet it’s different to write than to read)
- **Bulky**, since a storing element is “fat”; problematic to store a lot per unit area (compared to DRAM)
- **Expensive**, since it requires six transistors and special layout and support requirements
 - Ex: Assume you need a memory of 32 MB out of a 6B transistors budget
 - 1 B = 8 bits, 6 transistors per bit
 - # transistors for the memory: $32 \times 10^6 \times 8 \times 6 \approx 1.5 \times 10^9$
 - 1.5B transistors for this 32 MB memory!
 - 25% budget taken just for this 32 MB memory!



Type Memory: Dynamic RAM (DRAM)

- DRAM type memory: the information stored as a charge in a **capacitor**
 - Not charged (charged below a threshold value): 0 state
 - Charged (charged above a threshold value): 1 state
- The good: **cheap**, requires only one capacitor and one transistor
- The bad:
 - Capacitors leak, so the charge or lack of charge should be reinforced dynamically
 - While designed to store charge, aren't perfectly insulated (e.g., imperfect dielectric, aging)
 - Requires constant refreshing of the data stored in it to prevent information loss – this has overhead!
 - This is why we call it “dynamic” RAM



Comparison between SRAM and DRAM

- Order of the SRAM access time: **4 to 5 cycles**
 - Expensive but fast
 - Needs no refresh – data is kept in flip-flop as long as the device has power
- Order of the DRAM access time: **tens of cycles**
 - Less expensive but slow
 - Higher capacity per unit area
 - Needs refresh every 10-100 ms (thus sensitive to disturbances)

	Transistors per bit	Access Time	Persistent?	Sensitive?	Price	Applications
SRAM	6	1X	Yes	No	100X	Cache memories
DRAM	1	10s X	No	Yes	1X	Main Memory

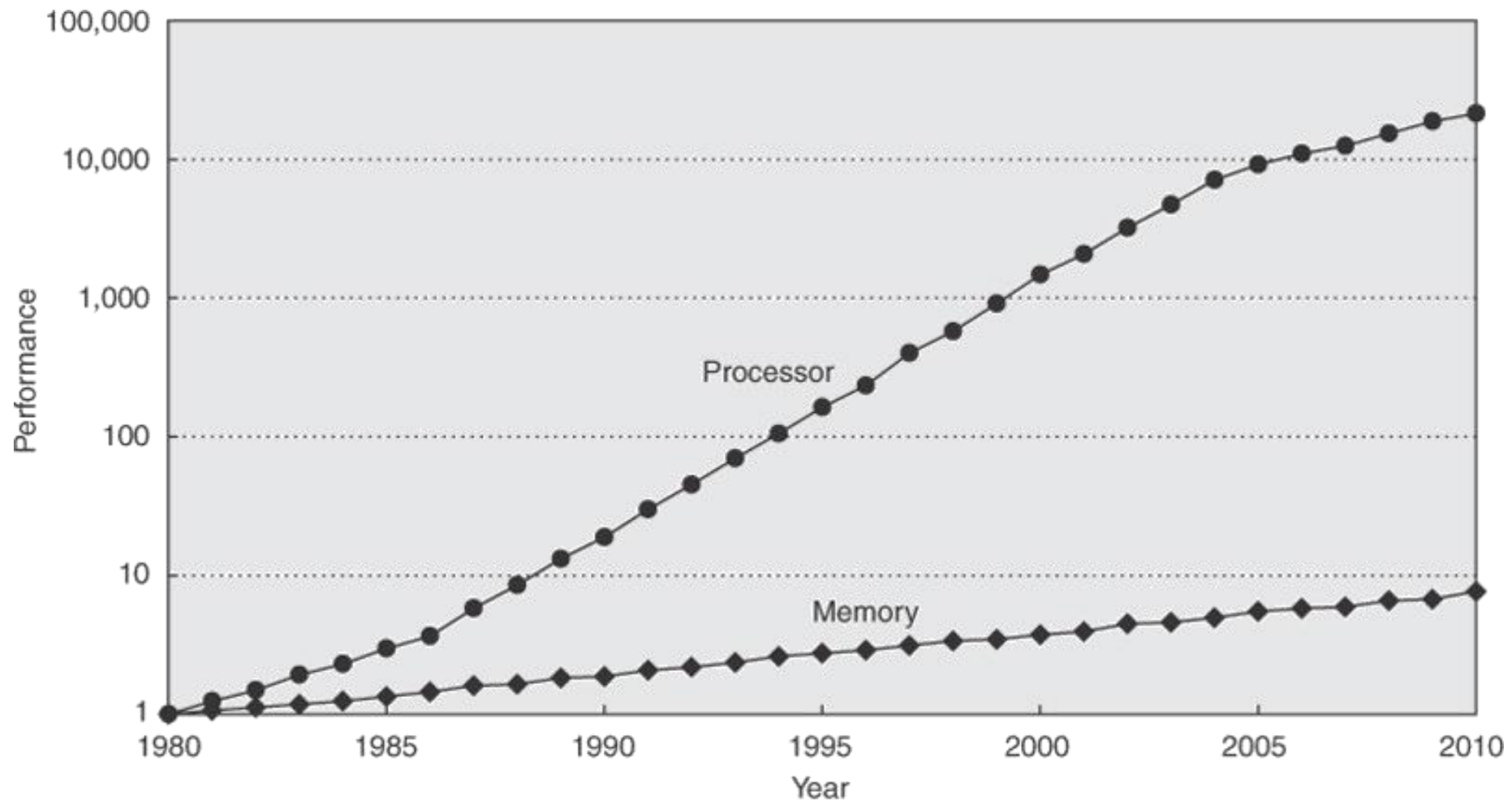
Feature Comparison Between Memory Types

	SRAM	DRAM	Flash ¹
Speed	Very fast	Fast	Very slow
Density	Low	High	Very high
Power	Low	High	Very low
Refresh	No	Yes	No
Retention	Volatile	Volatile	Non-volatile
Mechanism	Bi-stable Latch	Capacitor	Fowler-Nordheim tunneling

¹: Flash memory use: solid-state drives, USB flash drives, digital cameras

Memory Wall: Speed Gap between Processor and DRAM

Over time, while CPU processing speeds have increased dramatically, DRAM speed improvements have lagged behind (due to its physical nature and constraints), resulting to a bottleneck in system performance – a fact referred to as “memory wall”



Observation from the Memory Wall

- **Observation 1:** over time, speed of data processing increased faster than speed at which we could bring over data to be processed
 - DRAM access speed is not catching up with processor speed
- **Observation 2:** SRAM shown to eat lots of transistors and have large real-estate footprint
- **Observation 3:** the bigger the memory used to store data, the longer it takes to find/access an entry
 - Compromise between size and speed of access

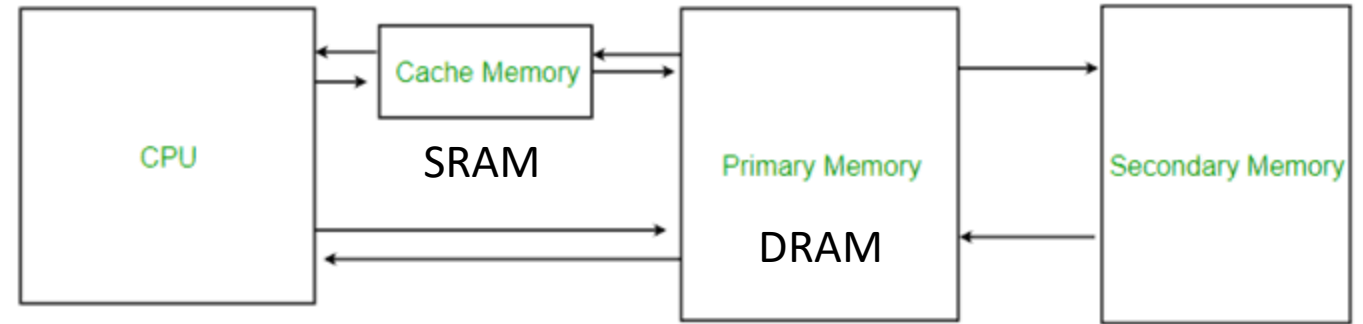
Consequences of common-sense observations

- Consequences of Observations 1 & 2 & 3: we need a **hierarchy of memories**
 - To strike a good trade-off between cost and performance
 - It took a while to shape up the hierarchy, happening through an evolutionary and continuous improvement process over computer architecture design
 - Managing memory hierarchy not trivial → a dedicated hardware asset called **MMU (memory management unit)** handling this task
 - **MMU's duties**: virtual memory management, cache control, memory protection, bus arbitration, etc.

The memory hierarchy, how it shapes up

- Since SRAM is expensive and bulkier, can't have too much – despite being fast in data access
 - Plagued by space & cost constraints

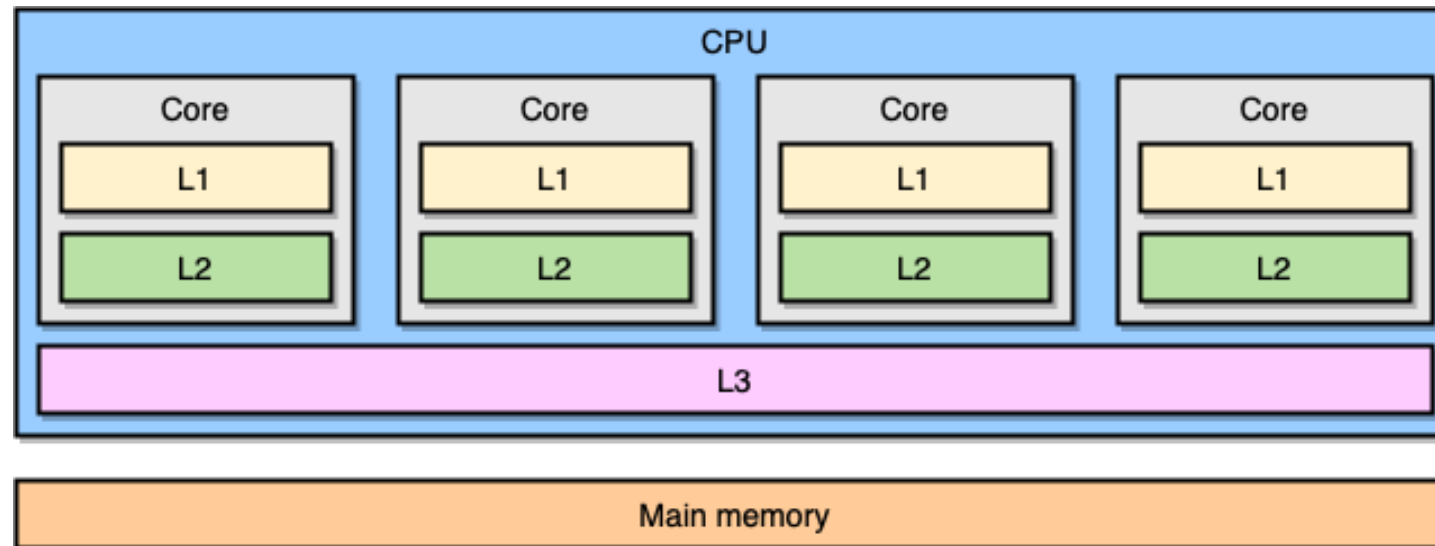
Simplest memory hierarchy:
Cache + Main memory



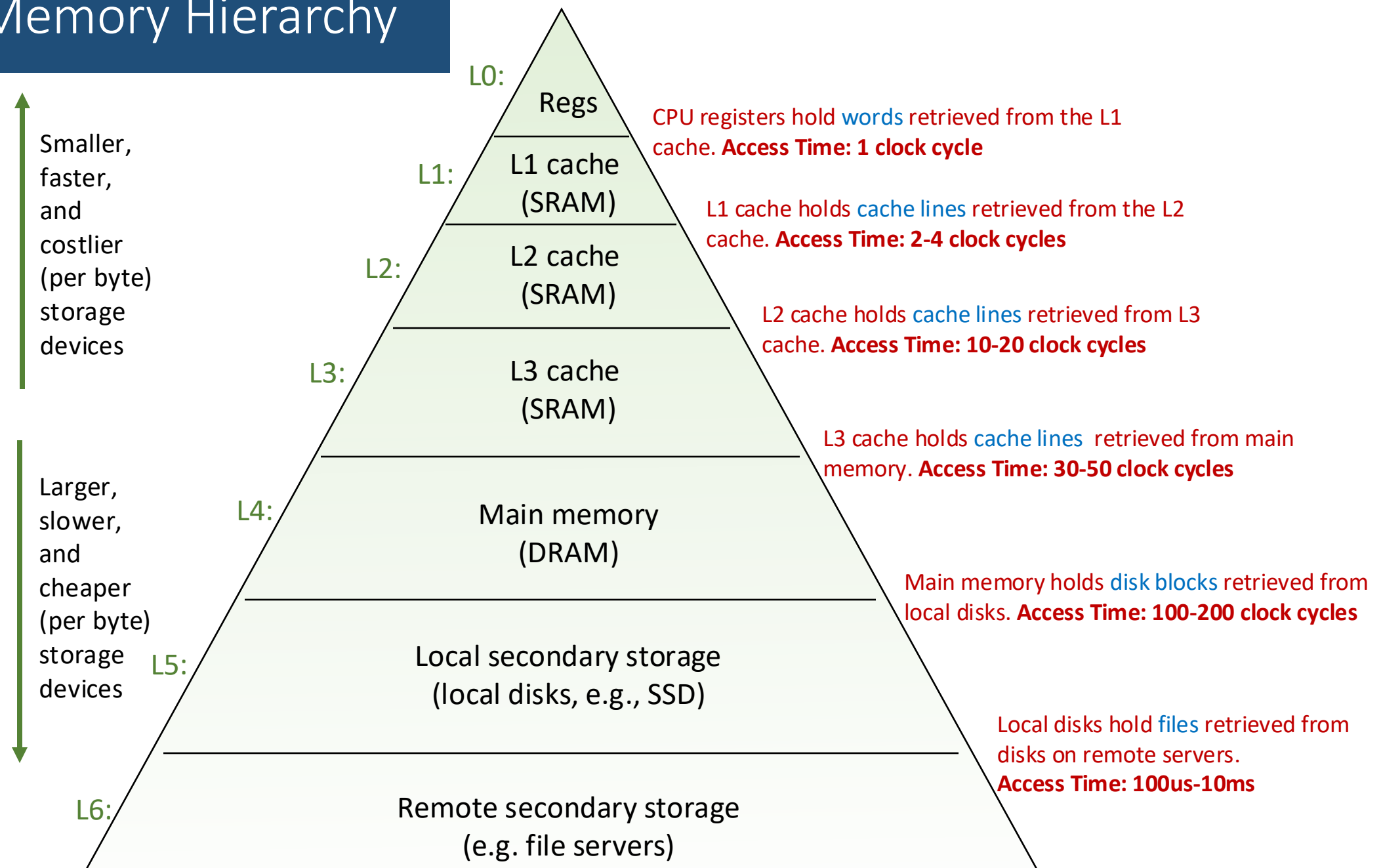
- Compromise:
 - Have some SRAM on-chip, making up what is called the “**cache**”
 - This is what has been used most frequently by the CPU (e.g., like your wallet)
 - Have a lot of inexpensive DRAM off-chip, making up the “**main (primary) memory**”
 - This is what people have in mind when they ask you “How much memory does your system have”?
- **To make a program run faster**, we always want to have low “average memory access time” by hitting the **cache** repeatedly instead of taking costly trips to **main memory**

There's Even a Hierarchy within Cache ...

- Today's memory architectures typically have multi-level caches: L1+L2+L3 caches
 - Level L1 faster and smaller than level L2 cache
 - L2 faster and smaller than L3
- All caches are typically on the chip and are SRAM

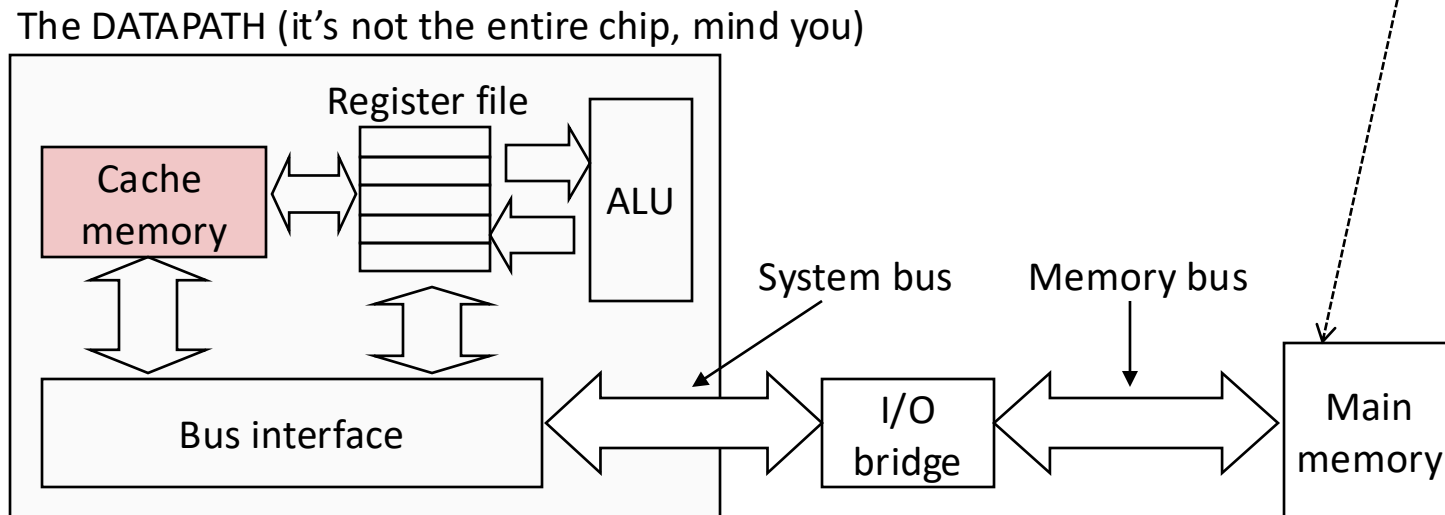


Typical Memory Hierarchy



Cache Memories

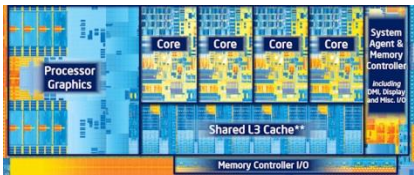
- **Cache memories** are small, fast, **on-chip** SRAM-based memories
 - Purpose: to store close to the CU/ALU frequently accessed blocks of main (aka “system”) memory
- Schematic of chip’s datapath to see where the cache fits in the big pic:



What it really looks like...

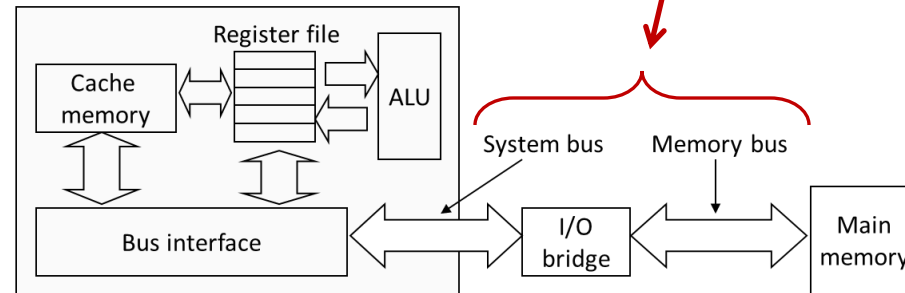
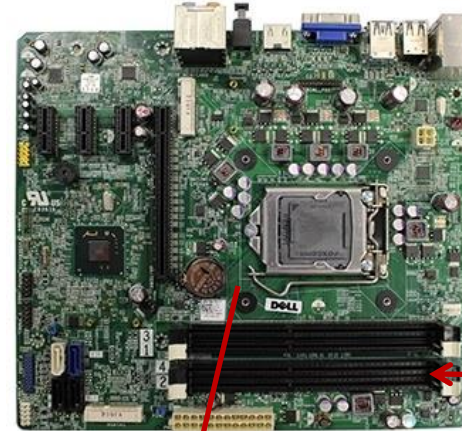


Source: PC Magazine



Source: techreport.com

Motherboard *Source: Dell*



Gets dropped in here

Main memory (DRAM)



Source: Dell

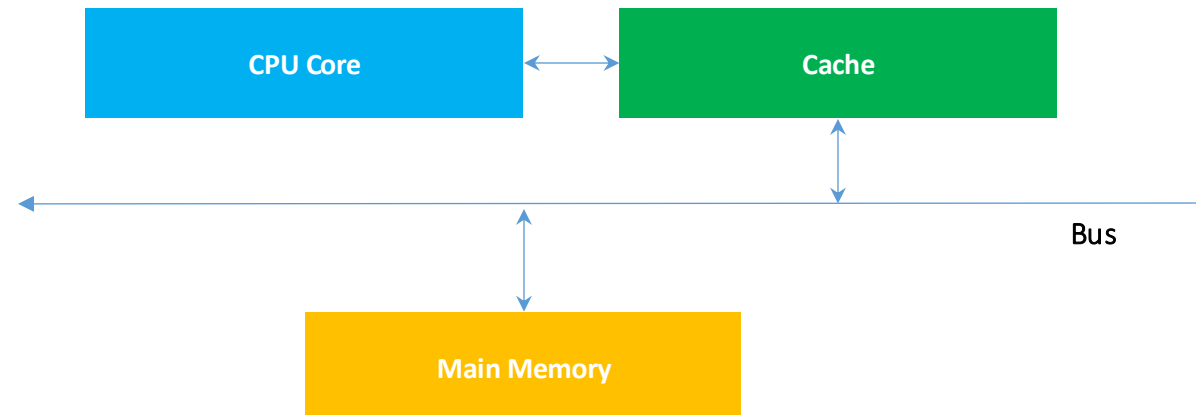
Desktop PC



Source: Dell

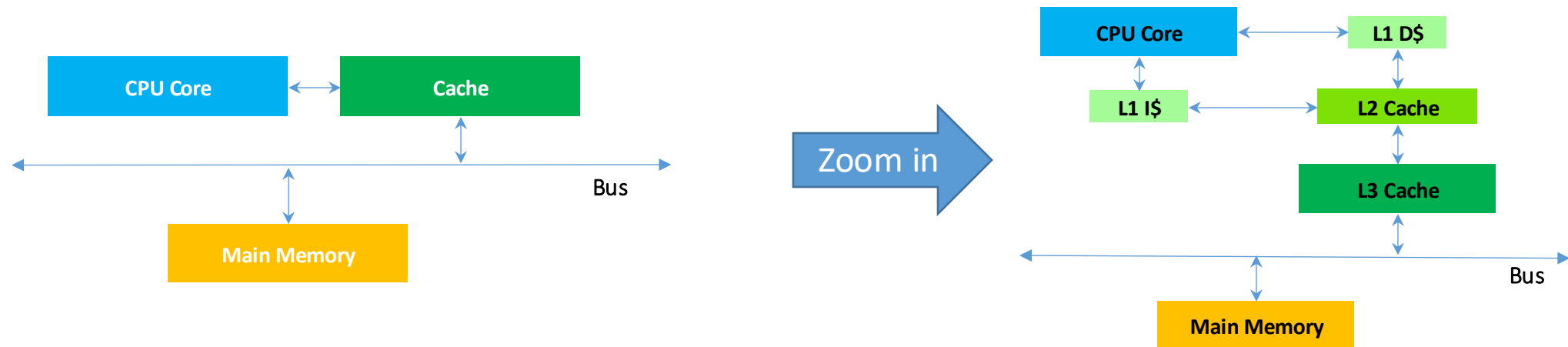
How Cache Comes into Play

- All* read/write operations eventually go through cache
 - This is where data gets stored before sending to registers processed through CU and ALU
- In general, we can have different cache partitions for holding data and instructions
 - Intel: L1 DC (data cache) and L1 IC (instruction cache) have been distinct since about 1993



Multiple Levels of Cache

- Typically, commodity chips today have 3 cache levels: L1, L2, and L3

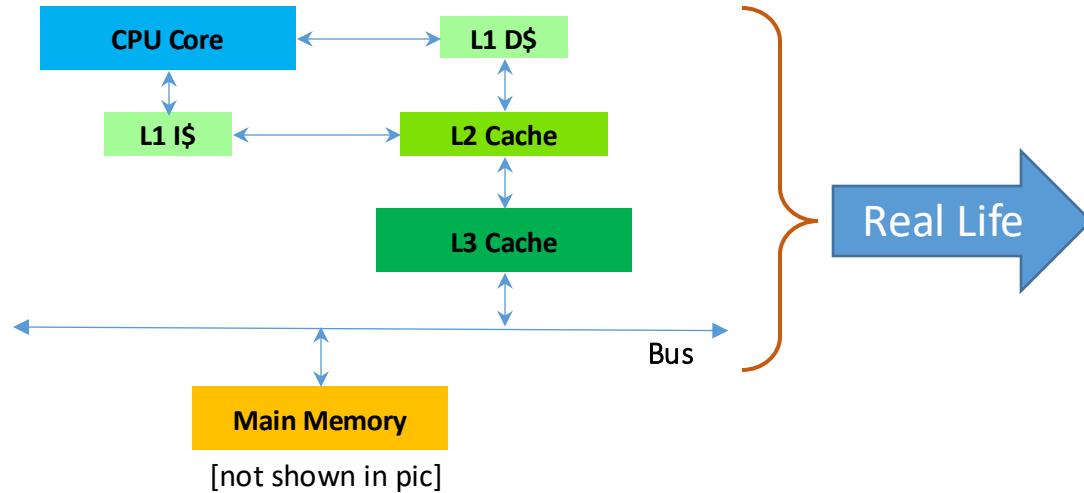


How cache looks in a real chip

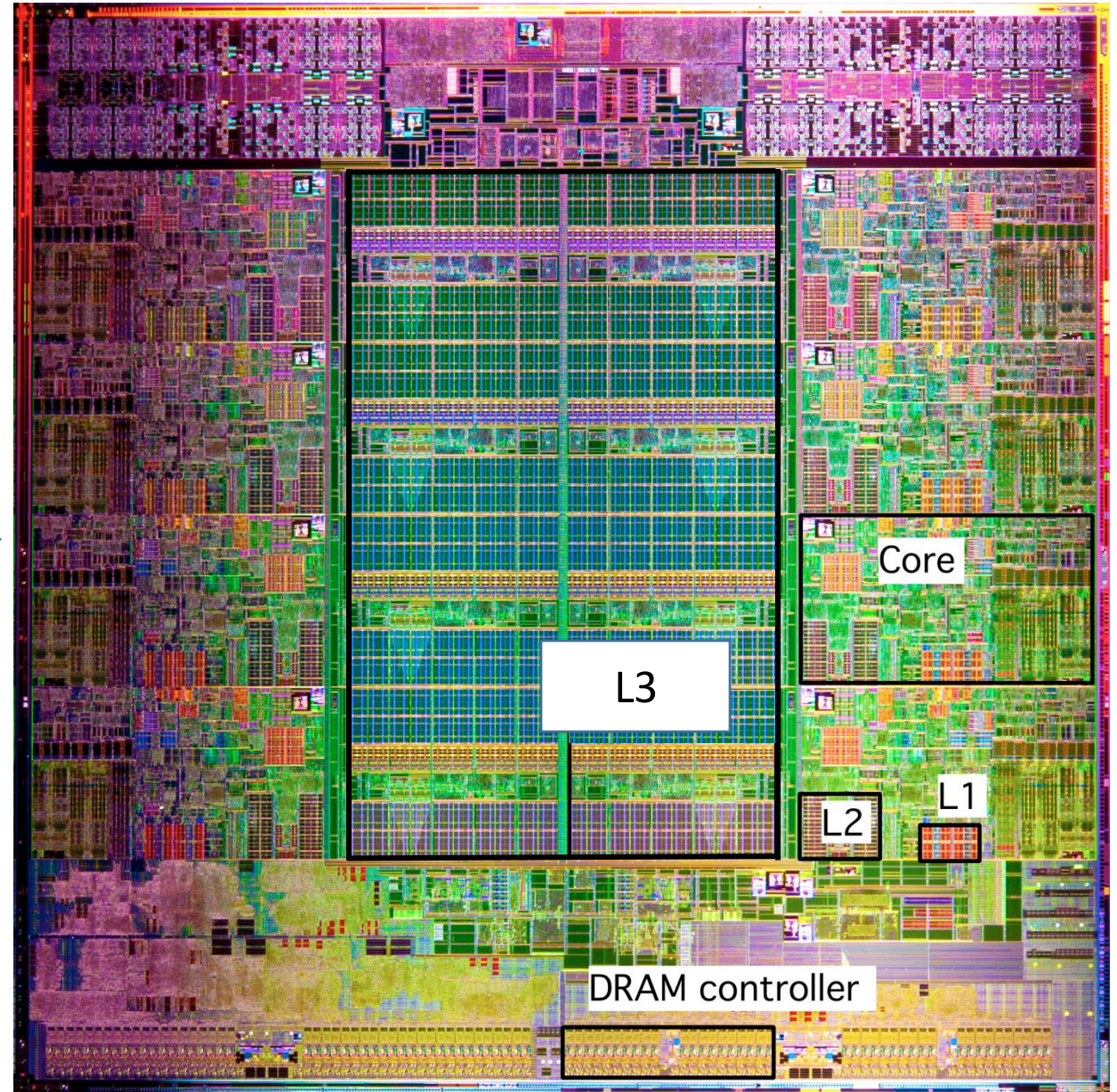


Source: PC Magazine

Intel Sandy Bridge-E
Processor Die
[8 core-version]

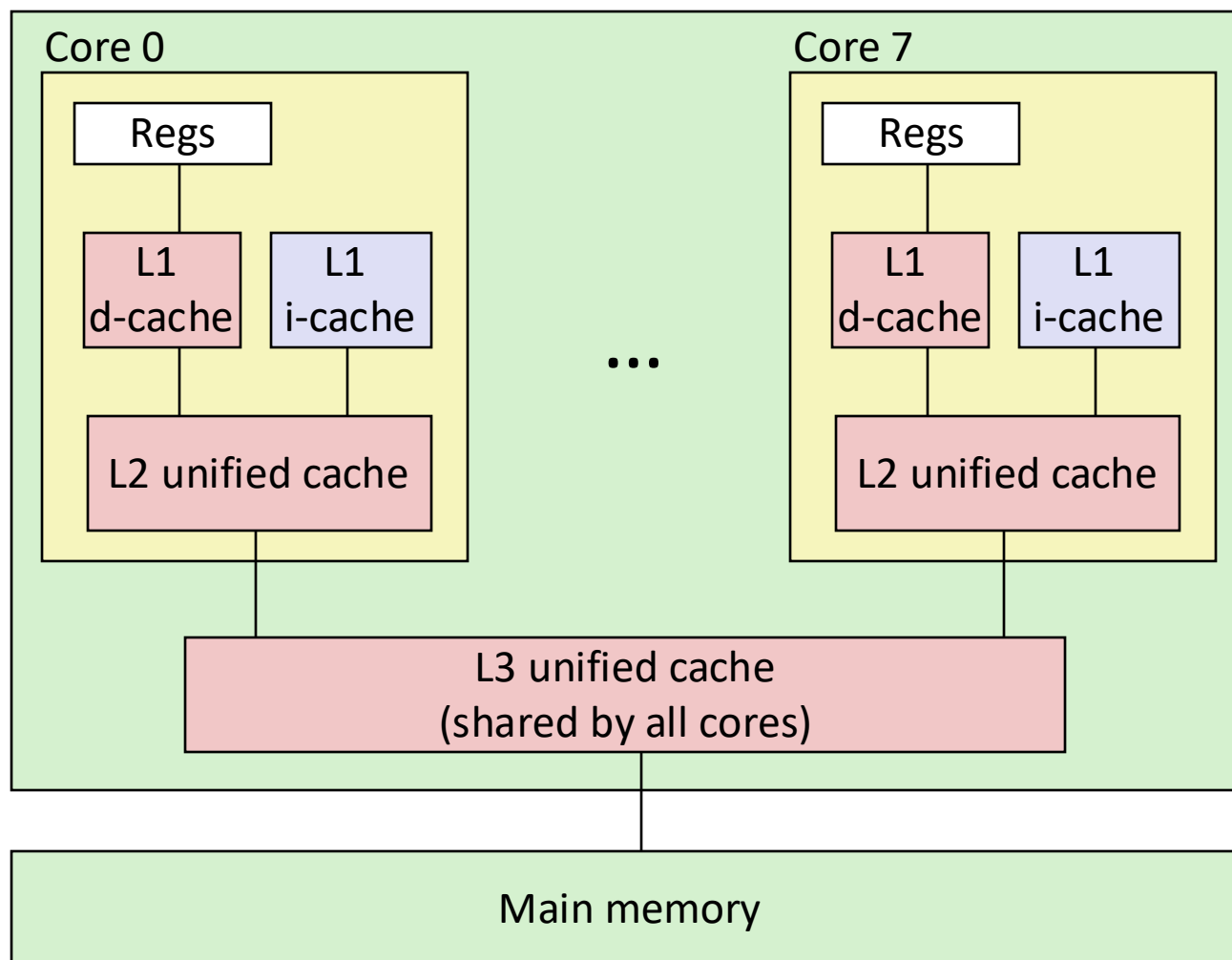


L1: 32KB Instruction + 32KB Data
L2: 256KB
L3: 5–20MB



Example: Intel Core i7 Cache Hierarchy

Processor package



L1 i-cache and d-cache:
32 KB, 8-way assoc.
Access: 4 cycles

L2 unified cache:
256 KB, 8-way assoc.
Access: 10 cycles

L3 unified cache:
8 MB, 16-way assoc.
Access: 40-75 cycles

Cache line size: 64 bytes
for all caches.

Usually, the cache is divided to two categories: IC vs DC

- **Instruction caches (ICs) store instructions**

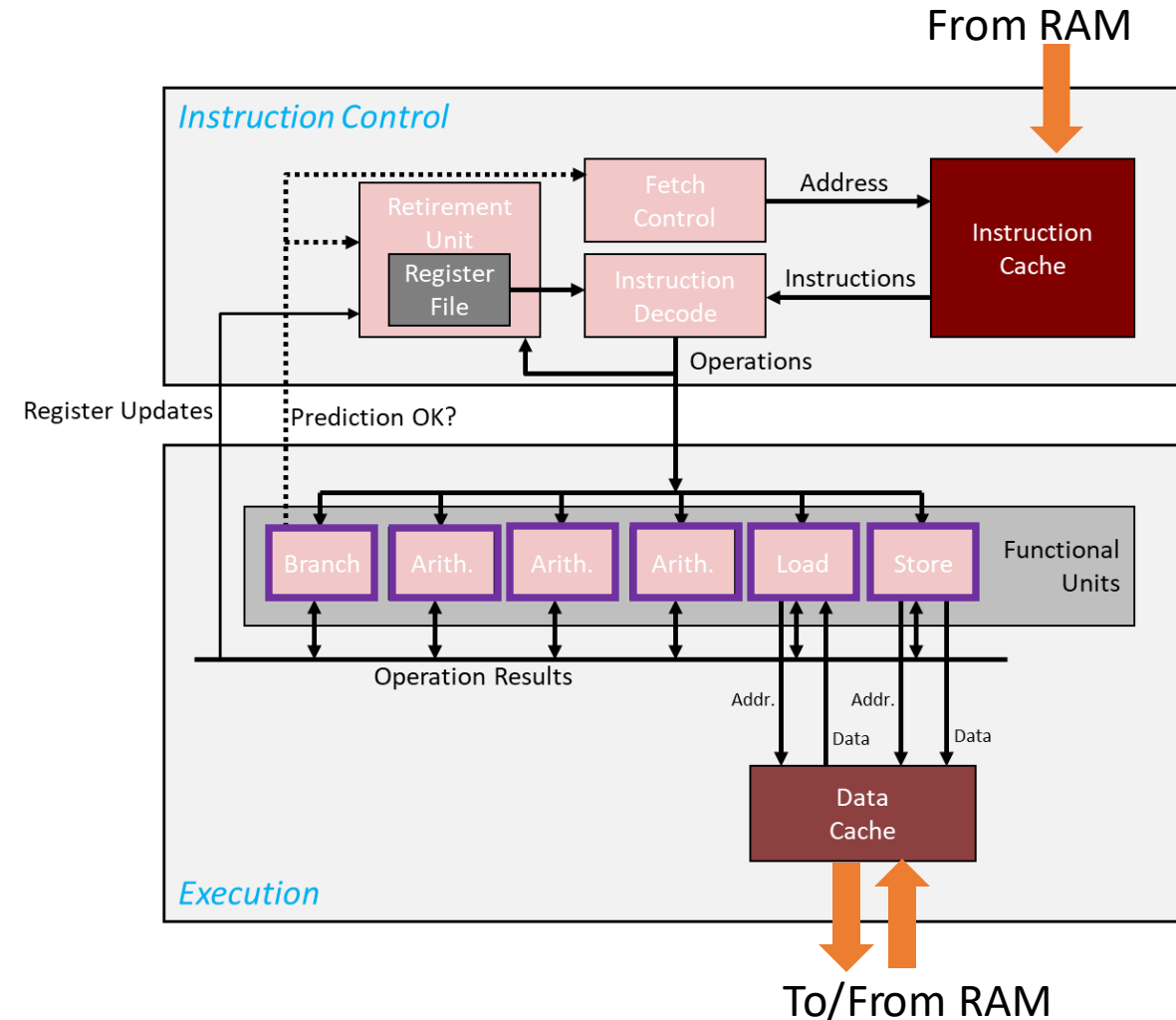
- Simpler to deal with compared to the data caches
- In general, instruction use is much more predictable than data use

- **Data caches (DCs) feed processor with data manipulated during execution**

- If processor would rely on data provided by main memory, the execution would be very slow
 - Ex: Main memory access time is 10-100x slower than L1 cache
 - Analog: every time you need money, would it be faster to take it from your wallet or to to bank?
- Also, “**Processor Clock**” operates much faster than the “**Memory Clock**”
 - Caches alleviate the memory pressure yet at the cost of processors being data hungry

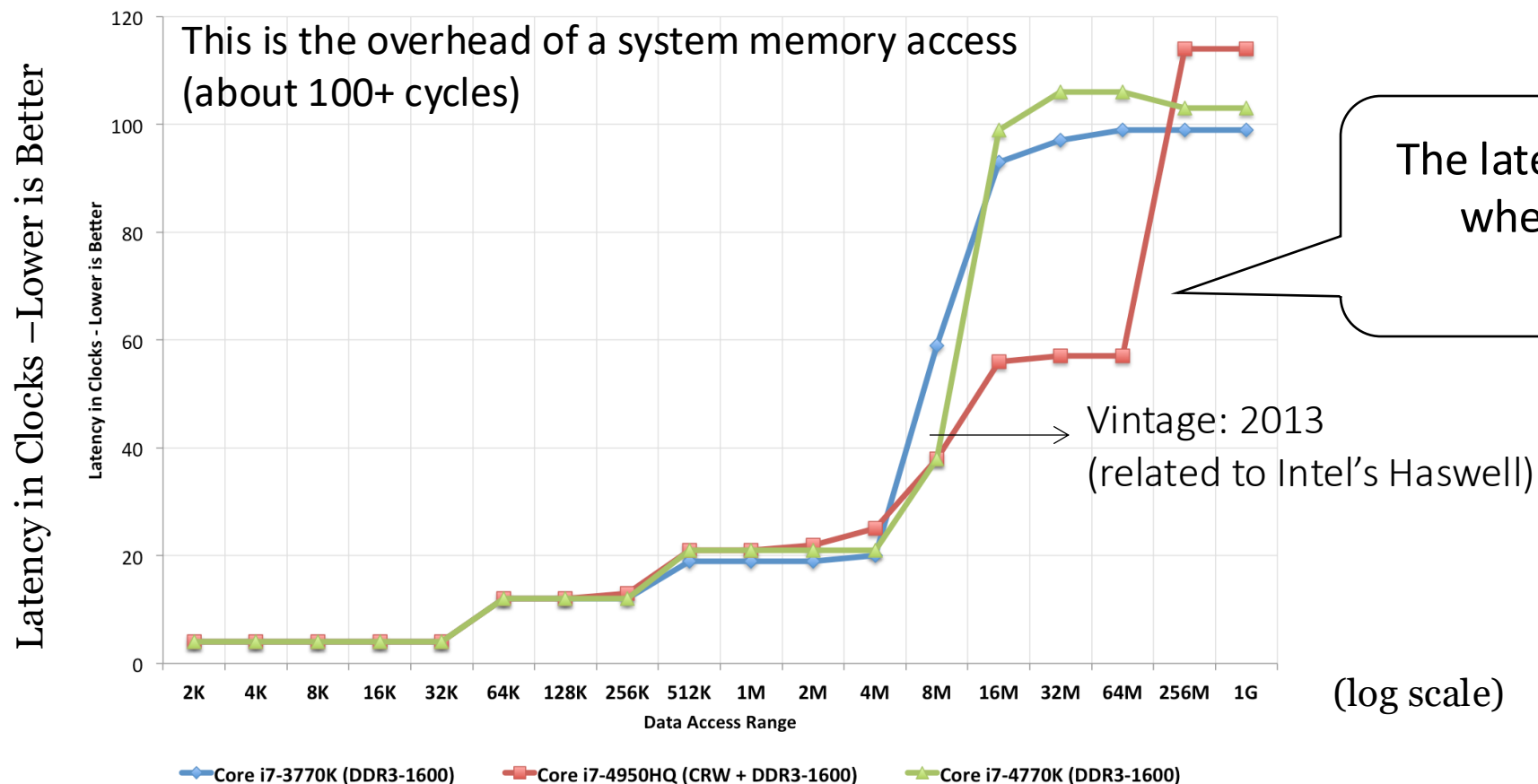
Processor Clock vs. Memory Clock [important to understand]

- Processor Clock: (look for this → )
 - Defines the frequency at which Functional Units operate
 - These functional units are “on-chip”
- Memory Clock: (look for this → )
 - Defines the frequency used to move data into/out from main memory into cache



Latency vs Data Access Range

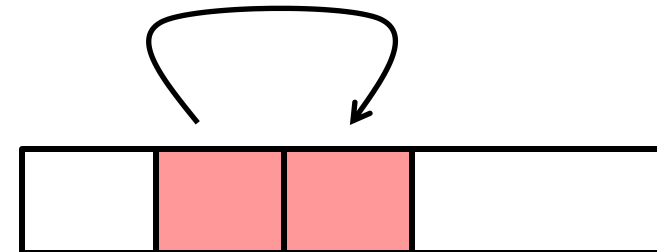
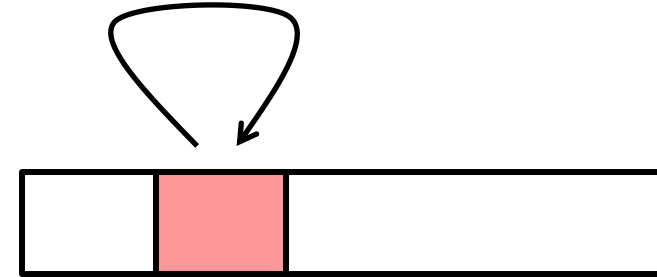
- Ratio between cache size and main memory size is very huge (like 1:1000)
 - 2013: 4 MB of L3 cache, versus 4 GB of main memory
 - 2017: 55 MB of L3 cache, versus 64 GB of main memory



The latency increases dramatically when the data access range becomes bigger!

We Want Locality to Make the Most of Cache

- **Principle of Locality:** Programs tend to use data and instructions with addresses near or equal to those they have used recently
- **Temporal locality:**
 - “Recently referenced items are likely to be referenced again in the near future”
- **Spatial locality:**
 - “Items with nearby addresses tend to come into use together”



Programs with locality often outperform those without locality

- *Temporal Locality* for memory access by a program
 - Idea: If you access a variable at some time, then you'll probably keep accessing the same variable for a while
 - Example: a for loop with some variables inside the loop → you keep accessing those variables as long as you loop
- *Spatial Locality* for memory access by a program
 - Idea: A memory access pattern characterized by bursts of repeated requests for data that is physically located within the same memory region
 - Example: a for loop with accessed data coming from the same distance difference → the data accessed between loop steps are adjacent to each other

Example regarding the concept of locality

```
sum = 0;  
for (i = 0; i < n; i++)  
    sum += abc[i];  
return sum;
```

- Looking at how the data is accessed:
 - Array `abc` elements are accessed in *stride-1* reference pattern (adjacent iterations differ by 1) → **spatial locality**
 - Variable `sum` is referenced at each iteration → **temporal locality**
- Instruction references
 - Reference instructions in sequence → **spatial locality**
 - Cycle through loop repeatedly → **temporal locality**

Quiz: What Locality can You Observe from this Example?

```
int main() {  
    const int size = 100;  
    int arr[size];  
  
    // Initialize the array with values  
    for (int i = 0; i < size; ++i) {  
        arr[i] = i;  
    }  
  
    // Access consecutive elements once  
    for (int i = 0; i < size; ++i) {  
        std::cout << arr[i] << " ";  
    }  
  
    return 0;  
}
```

Quiz: What Locality can You Observe from this Another Example?

```
const int size = 10000;
std::vector<int> arr(size);

// Initialize the array with random values
for (int i = 0; i < size; ++i) {
    arr[i] = rand() % size;
}

// Select a set of random indices to access repeatedly
std::vector<int> indices = {135, 2997, 4523, 8761, 6221, 1453};

// Access only the elements at the above 6 indices repeatedly
for (int i = 0; i < 1000000; ++i) {
    for (int idx : indices) {
        arr[idx]++;
    }
}
```